Budget-Constrained Non-Stationary LLM Routing via Primal-Dual Contextual Bandits

Abstract

Large Language Model (LLM) routing dynamically directs user queries across a portfolio of models—from lightweight open weights to frontier APIs—optimizing the trade-off between inference cost and response quality. While static classifiers are commonly deployed for this task, production environments exhibit significant non-stationarity: models receive silent performance updates, inference providers drop prices abruptly, and aggregate API limits impose strict dollar budgets. In this paper, we present an open-source framework for non-stationary LLM routing via cost-aware contextual bandits. We demonstrate three key capabilities critical for production deployments: (1) an online Lagrangian relaxation (Primal-Dual Contextual Bandits with Knapsacks) that enforces strict average-cost budgets dynamically, eliminating the need for offline hyperparameter tuning of cost penalties; (2) a geometric forgetting mechanism that enables the router to adapt gracefully to model quality shifts without suffering catastrophic forgetting of its initial offline-trained priors; and (3) the application of a rigorous hyperparameter tuning methodology to LLM routing that jointly optimizes exploration, prior strength, and forgetting factors. While established in other domains, adapting this tuning specifically addresses the uniquely contending requirements of LLM routing, allowing the system to strike an optimal balance between stable stationary exploitation and rapid agility under distribution shifts.

Through empirical evaluation on an established dataset of 1,785 realistic prompts, spanning a portfolio with a 500× cost differential (Llama-3.1-8B to Gemini-2.5-Pro), we demonstrate our system's robustness. Under simulated reward drift, our approach significantly outperforms both frozen static policies and naive infinite-memory bandits. Critically, we demonstrate that when cost drift (e.g., a mid-cycle price drop) and budget constraints interact, our adaptive budget pacer automatically exploits the freed capital to route towards higher-quality frontier models, improving overall application quality while remaining strictly budget-compliant. We release the fully documented routing framework to the open-source community to facilitate both further research and immediate production use.

ACM Reference Format:

. 2026. Budget-Constrained Non-Stationary LLM Routing via Primal-Dual Contextual Bandits. In . ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/1122445.1122456

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/1122445.1122456

1 Introduction

Large language model (LLM) inference increasingly draws from multi-provider portfolios [3], spanning lightweight open-weights models to frontier APIs. Because the optimal model varies across tasks, domains, and individual prompts, **LLM routing**—dynamically selecting the best model per request—has emerged as a critical lever for managing cost and quality at scale [5, 6, 11, 12]. In production, an engineering team may route millions of requests daily across models spanning a 500× price range (e.g., $0.00003/req for Llama-3.1-8B vs. $0.015/req for Gemini-2.5-Pro) and must keep average spend within a strict API budget.

Many existing routers operate as static classifiers trained offline on evaluation data [5, 6, 11]. While highly effective in stationary environments, frozen policies risk degradation when deployed in real-world scenarios due to three critical factors. First, **deployment distributions drift**: prompt mixes vary by use case and drift over time from offline training data [2]. Second, **model quality is non-stationary**: providers silently update APIs in ways that can subtly shift downstream performance on specific tasks [4, 10], meaning a router trained on historical data inherits a *stale policy*. Third, **model costs drop frequently**: rapid frontier releases often trigger price cuts [14]. If a static router maintains a fixed routing distribution during a price drop, it merely pockets the savings, leaving potential quality gains—such as upgrading a larger percentage of traffic to the now-cheaper frontier model—unexploited.

To address the need for online adaptation, recent work has formulated routing as a contextual bandit or reinforcement learning problem [9, 12, 13, 15]. These approaches learn dynamically from feedback (e.g., user ratings or automated judge scores) to adjust their policies over time. However, three critical challenges remain for deploying these adaptive systems in budget-constrained enterprise environments: 1. **Budget Enforcement**: Existing cost-aware bandits typically scalarize cost into the reward via a static penalty weight λ [11, 15]. This requires offline tuning to translate an abstract weight into a dollar outcome, and must be manually re-tuned whenever pricing changes or the traffic distribution shifts. 2. **Catastrophic Memory vs. Agility**: Standard algorithms like LinUCB often use infinite memory (forgetting factor $\gamma = 1.0$). While optimal for stationary settings, this causes the bandit to react far too slowly when a previously high-performing model suddenly degrades in quality, accumulating massive regret during the transition. 3. **Contending Contribution Requirements**: Tuning an adaptive system to simultaneously maintain stable stationary performance (which favors infinite memory and low exploration) and agile non-stationary adaptation (which favors aggressive forgetting and high exploration) creates conflicting optimization objectives. Standard cross-validation optimizes solely for stationary metrics, leading to brittle real-world deployments.

We address these practical deployment challenges with **BanditGPT**, an open-source contextual bandit framework explicitly designed for budget-constrained, non-stationary LLM routing. We make four key contributions, evaluated on a dataset of 1,785 prompts

scored by a continuous three-factor rubric (cross-judge robustness validated in Appendix **??**) across a canonical three-tier model portfolio:

(1) **Direct Budget Pacing via Primal-Dual CBwK:** We formulate routing as a Contextual Bandit with Knapsacks (CBwK) and implement an online Lagrangian relaxation (`BudgetPacer`) that accepts a direct dollar-budget target. By applying exponential moving average (EMA) smoothing to handle the extreme variance in LLM pricing, our system dynamically adjusts penalties to achieve exact budget compliance while automatically tracing the Pareto frontier, eliminating offline λ tuning.
(2) **Robust Adaptation via Geometric Forgetting:** To combat silent model updates (reward drift), we integrate a geometric forgetting mechanism ($\gamma = 0.995$) coupled with offline-to-online warmup priors. This enables the router to adapt rapidly to quality shifts without suffering catastrophic forgetting of the initial prompt-preference mapping, drastically reducing cumulative regret under drift compared to naive bandits ($\gamma = 1.0$) and frozen policies.
(3) **Exploiting Cost Drift under Budget Constraints:** We demonstrate that when a model provider drops its price mid-cycle, the `BudgetPacer` automatically detects and exploits the freed capital. Unlike static baselines that fail to adapt, our system dynamically reallocates traffic to higher-quality models, maximizing application quality while remaining strictly budget-compliant. We additionally analyze the limitations of adaptation at extremely restrictive budgets, termed the "exploration starvation" effect.
(4) **Rigorous Hyperparameter Optimization for Shift:** We adapt joint hyperparameter tuning (exploration α, prior strength n_{eff}, forgetting factor γ) to the LLM routing domain. While utilized in other fields, this approach directly resolves the uniquely contending requirements of our system, structurally balancing stable stationary exploitation with rapid agility under distribution shifts where standard cross-validation fails.

By releasing the framework open-source, we aim to provide an immediate pathway for practitioners managing enterprise LLM gateways to replace manual recalibration with autonomous Primal-Dual pacing, and to offer the research community a reproducible testbed for non-stationary routing. The remainder of this paper is organized as follows: Section 2 surveys related work; Section 3 formulates the budget-constrained non-stationary routing problem; Section 4 presents the system architecture; Section 5 details the experimental setup; Section 6 discusses the empirical evaluation; and Section 7 outlines practical deployment considerations.

2 Related Work

LLM routing has evolved rapidly from cascading heuristics to supervised classifiers, and more recently, to adaptive routing systems. In this section, we situate BanditGPT within the landscape of these approaches, highlighting the gaps in addressing production non-stationarity and dynamic budget constraints.

2.1 Static and Supervised Routers

Early approaches to cost-aware LLM routing relied on static rules or cascading heuristics (e.g., FrugalGPT [3]), where a system queries models sequentially from cheapest to most expensive, stopping when a predicted quality threshold is met. While simple to implement, cascades introduce highly variable latency, as complex queries must await sequential failures before reaching a capable model.

More recent systems formulate routing as a supervised classification task trained offline on preference or evaluation data. RouteLLM [11] uses matrix factorization on pairwise preference data to map queries to models via a cost threshold. GraphRouter [6] constructs a heterogeneous graph over tasks, queries, and LLMs, using edge prediction to estimate per-model quality and cost, allowing it to generalize to newly added LLMs without full retraining. RouterDC [5] learns joint embeddings of queries and LLMs via dual contrastive losses, routing to the model with the highest similarity score.

While supervised routers achieve strong performance when deployed on stationary distributions that match their training data, they exhibit two critical flaws in production environments. First, they deploy *frozen policies* that cannot adapt to non-stationarity (e.g., silent model updates or shifting user prompts). Second, they enforce budget constraints via fixed, offline-tuned thresholds or cost penalties. If a model provider unexpectedly drops prices, these static systems fail to dynamically exploit the new cost-quality trade-off, either under-spending their budget or requiring a manual, offline retraining and recalibration cycle.

2.2 Adaptive and Bandit-Based Routers

To enable online adaptation, a recent wave of research has formulated LLM routing as a contextual bandit or reinforcement learning (RL) problem, allowing systems to learn continuously from deployment feedback.

PILOT [12] extends LinUCB [8] with *preference-prior informed* initialization, injecting LLM embeddings learned offline from human preference data to mitigate the cold-start problem. To manage costs, PILOT employs an online multi-choice knapsack algorithm to optimize cumulative cost over a known, finite horizon. While theoretically grounded, the knapsack approach requires knowing the total number of queries in advance, making it difficult to apply to open-ended, continuous production streams where request volume fluctuates.

BaRP [15] trains a contextual bandit using entropy-regularized policy gradients, conditioning routing on a user preference vector to allow inference-time adjustments to the cost-quality trade-off. LLM Bandit [9] utilizes a multi-armed bandit with preference-conditioned routing, incorporating model identity vectors to generalize to unseen models. Similarly, xRouter [13] takes an RL approach, fine-tuning a small language model to act as both a router and a responder, trained end-to-end with a cost-aware reward signal. Finally, recent work on contextual queueing bandits (e.g., ACQB [**?**]) leverages implicit feedback from user retrials to jointly optimize routing and scheduling in conversational services.

2.3 Positioning BanditGPT

BanditGPT builds upon the foundation of contextual bandits but introduces critical structural advances designed specifically for the

realities of open-ended, budget-constrained production environments:

(1) **Continuous Budget Pacing:** Unlike PILOT's finite-horizon knapsack or the static scalarized cost penalties used in RouteLLM and BaRP, BanditGPT utilizes a Primal-Dual Contextual Bandit with Knapsacks (CBwK) framework. This *BudgetPacer* strictly enforces a per-request average cost target (e.g., "$0.005 per request") over an infinite horizon, automatically adjusting to price drops or traffic shifts without offline recalibration.
(2) **Explicit Handling of Non-Stationarity:** While existing bandit routers continuously learn, they often employ infinite memory (e.g., standard LinUCB). When model quality degrades (reward drift), infinite memory causes catastrophic inertia, as the router slowly unlearns thousands of stale historical observations. BanditGPT introduces a principled geometric forgetting mechanism that dynamically discounts stale evidence, enabling rapid adaptation to quality shifts while preserving enough historical context to avoid volatile, uninformed routing.

3 Problem Formulation

We formalize LLM routing as an online decision-making problem under budget constraints, where the environment is non-stationary with respect to both model quality (reward drift) and model pricing (cost drift).

3.1 Contextual Bandit Routing

Let $t \in \{1, 2, \ldots, T\}$ denote a discrete sequence of user requests (prompts). At each step t, the router receives a context vector $x_t \in \mathbb{R}^d$ representing the prompt, derived from a pre-trained embedding model (e.g., SentenceTransformers). The router must select a single language model a_t from a discrete portfolio of available models $\mathcal{A} = \{1, \ldots, K\}$.

Upon executing the prompt on model a_t, the system observes a scalar reward $r_t \in [0, 1]$ indicating the quality of the response. This reward may be derived from an automated LLM-as-a-judge, explicit user feedback (e.g., thumbs up/down), or an implicit task completion metric. Crucially, the router operates under *bandit feedback*: it only observes the reward for the chosen model a_t and does not know what the counterfactual rewards would have been for the unselected models.

The primary objective is to learn a routing policy π that maximizes the expected cumulative reward:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=1}^{T} r_{t,\pi(x_t)}\right] \tag{1}$$

3.2 The Budget Constraint

In production, each model $a \in \mathcal{A}$ incurs a known, deterministic inference cost $c_a(x_t)$ (typically priced per input and output token). Because the models in a portfolio often span orders of magnitude in price—from open-weights models costing fractions of a cent to frontier APIs costing several cents per request—the router must strictly manage aggregate spend.

We formulate this as a *Contextual Bandit with Knapsacks* (CBwK) [1] problem. The system is given a target average budget B (e.g., "average $0.005 per req"). The policy π must maximize cumulative reward subject to the constraint that the long-run average cost does not exceed B:

$$\limsup_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} c_{\pi(x_t)}(x_t) \le B \tag{2}$$

The standard approach to this constraint in the routing literature is to scalarize the reward using a fixed penalty weight $\lambda > 0$, optimizing a penalized utility $u_{a,t} = r_{a,t} - \lambda c_a(x_t)$ [11]. However, selecting λ requires extensive offline simulation on historical data to find the weight that happens to yield an average cost of B. If the distribution of prompts shifts, or if the portfolio changes, the realized cost will drift away from B, requiring manual recalibration.

3.3 Non-Stationarity

Real-world LLM deployments are fundamentally non-stationary. We define two specific types of environment drift that a production router must handle autonomously:

Reward Drift (Quality Shifts). LLM providers silently update their models behind APIs, or internal teams push new versions of fine-tuned open-weight models. Formally, the conditional expected reward function $f_a(x) = \mathbb{E}[r_t \mid x, a]$ changes at some unknown time τ, such that $f_a^{(t<\tau)}(x) \neq f_a^{(t\ge\tau)}(x)$. A router must detect this shift and rapidly unlearn its historical estimates for model a, avoiding a stale policy that continues routing traffic to a degraded model.

Cost Drift (Price Drops). API providers frequently reduce inference prices to remain competitive [14]. Formally, the cost function $c_a(x)$ decreases at time τ. If the router uses a fixed, offline-tuned penalty λ, it will continue to apply the same relative penalty to model a. While this mechanically reduces the total spend (pocketing the savings), it fails to recognize that model a is now a relatively better value. An optimal router should dynamically exploit the newly freed budget to route more challenging queries to the newly-cheaper, higher-quality model, maximizing overall application quality while remaining precisely at the budget limit B.

4 Methodology: The BanditGPT System

We present the architecture of BanditGPT, an open-source contextual bandit routing framework. This section details the end-to-end routing pipeline, our Primal-Dual budget pacing mechanism, and the geometric forgetting approach for non-stationary environments.

4.1 System Architecture Overview

BanditGPT operates as an asynchronous, closed-loop routing system (Figure 1). At inference time (step t), a user query x_t is processed by a lightweight feature extractor (e.g., a SentenceTransformer embedding model), which passes the query vector to the `BanditRouter`. The router maintains a set of sufficient statistics for each model $a \in \mathcal{A}$, initialized via *offline-to-online warmup priors* to mitigate the cold-start problem. The router computes a score for each model, incorporating uncertainty (LinUCB) and subtracting a dynamic cost penalty provided by the `BudgetPacer`. The router then selects the model a_t with the highest score, returning its response to the user.

After the query completes and is evaluated (either synchronously by an automated judge, or asynchronously via delayed user feedback), the reward r_t and exact cost c_t are fed back into the system. The `BanditRouter` updates its reward estimates (incorporating geometric forgetting), while the `BudgetPacer` updates its internal dual variable λ_t, adjusting the cost penalties for future requests to steer the system back toward the target budget B.

4.2 Primal-Dual Budget Pacing

To enforce strict, dynamically adjusting budget constraints without manual offline tuning of a static penalty weight λ_s, BanditGPT implements a Primal-Dual Contextual Bandit with Knapsacks (CBwK) formulation [1] via the `BudgetPacer`.

At each step t, the router selects model a_t by maximizing a budget-augmented utility function:

$$a_t = \arg\max_{a \in \mathcal{A}_t} \Big[\underbrace{\hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}}_{\text{LinUCB score [8]}} - \underbrace{\lambda_s \, \tilde{c}_a}_{\text{static penalty}} - \underbrace{\lambda_t \, \tilde{c}_a}_{\text{dual penalty}} \Big] \tag{3}$$

where $\mathcal{A}_t \subseteq \mathcal{A}$ is the set of models surviving a hard cost ceiling (detailed below), $\tilde{c}_a \in [0, 1]$ is the log-normalized unit cost, and λ_t is a Lagrange multiplier updated after observing the realized cost c_t.

Dual-Variable Dynamics and EMA Smoothing. The core innovation of the `BudgetPacer` is how it tracks and updates the dual variable λ_t in response to the extreme variance in LLM pricing. Rather than updating λ_t directly from the raw per-request cost c_t, the pacer computes an *EMA-smoothed* cost signal:

$$\bar{c}_t = (1 - \alpha_{\text{ema}}) \, \bar{c}_{t-1} + \alpha_{\text{ema}} \, c_t \tag{4}$$

$$\lambda_{t+1} = \min\Big(\bar{\lambda}, \ \max\big(0, \ \lambda_t + \eta \, (\bar{c}_t / B - 1)\big) \Big) \tag{5}$$

where B is the user's desired average cost per request, $\alpha_{\text{ema}} = 0.05$ is the smoothing factor, $\eta = 0.05$ is the step size, and $\bar{\lambda} = 5$ is a capacity cap.

Using the EMA $\bar{c}_t$ is critical. If a single \$0.015 Gemini request were routed, the raw ratio c_t / B could spike to 500× the target B (e.g., if B = \$0.00003). This would produce massive, sawtooth oscillations in λ_t that never converge, rendering the penalty useless. The EMA smooths this variance, generating a stable gradient signal whose magnitude reflects the *average* cost regime over recent history.

Furthermore, normalizing the gradient by B (i.e., $\bar{c}_t / B$) makes the step size η portfolio-independent. The same η produces equivalent λ_t trajectories whether the cheapest model costs \$0.00003 or \$0.03, eliminating the need to retune hyperparameters when the pricing scale changes.

Two-Layer Enforcement (Adaptive Mode). The `BudgetPacer` combines two enforcement mechanisms simultaneously:

- **Hard ceiling (Safety Net):** When $\lambda_t > 0$, the candidate set $\mathcal{A}_t$ actively excludes models whose blended unit price exceeds the dynamic ceiling. This acts as a circuit breaker, preventing catastrophic overspend on any single request when the budget is severely constrained.
- **Soft penalty (Optimizer):** The dual penalty $\lambda_t \tilde{c}_a$ biases the UCB score toward cheaper models proportionally to their cost. This allows fine-grained, context-dependent routing within the surviving candidate set, minimizing quality loss within the hard budget envelope.

4.3 Non-Stationary Adaptation via Geometric Forgetting

LLM routing in production is non-stationary: providers silently update models, creating reward drift. A naive LinUCB algorithm ($\gamma = 1.0$) accumulates infinite memory, severely slowing adaptation to a shifting reward landscape because it must overcome thousands of historical observations. Conversely, entirely discarding history discards the valuable prior knowledge required for stable, informed routing.

BanditGPT addresses non-stationarity through a *forgetting factor* $\gamma \in (0, 1]$ applied directly to the disjoint LinUCB sufficient statistics. At each step t, before incorporating the new observation (x_t, a_t, r_t), the design matrix and reward accumulator for the chosen arm a_t are exponentially discounted:

$$A_{a_t} \leftarrow \gamma \, A_{a_t} + x_t x_t^\top \tag{6}$$

$$b_{a_t} \leftarrow \gamma \, b_{a_t} + r_t \, x_t \tag{7}$$

This exponential weighting [7] gives observations an effective half-life of $\ln 2 / (1 - \gamma)$ steps. We empirically find that $\gamma = 0.995$ strikes an optimal balance, creating an effective memory window of approximately 200 steps (see Section 6.4 for the formal selection procedure). Recent evidence dominates, allowing the router to adapt to shifts rapidly, while historical observations (including the critical offline warmup priors) are geometrically discounted but not erased entirely.

The parameter γ thus controls a bias–variance tradeoff between *stability* (trusting historical evidence to maintain high baseline quality) and *plasticity* (responding quickly to new evidence when models degrade).

5 Experimental Setup

We evaluate BanditGPT on a canonical portfolio of three LLMs, spanning a large cost differential, and using realistic prompt data to simulate both stationary and non-stationary production environments. Table 1 summarises the data splits and model portfolio.

5.1 Datasets and Ground Truth

We collect 11,983 prompts from the LMSYS Chatbot Arena [16], reflecting real-world, open-ended user interactions spanning coding, reasoning, and conversational tasks. Each prompt is routed to all three models in the portfolio and evaluated by an automated LLM-as-a-judge.

Quality evaluation. We employ DeepSeek-R1 [**?**] as the evaluator. Each response is scored on three continuous dimensions—*Reasoning Quality* (40%), *Instruction Following* (30%), and *Communication Quality* (30%)—yielding a composite scalar reward $r_t \in [0, 1]$. This continuous rubric provides the granular quality distinctions necessary for multi-arm routing; binary win/loss outcomes collapse into ties on a large fraction of prompts and are insufficient for $K > 2$ portfolios.

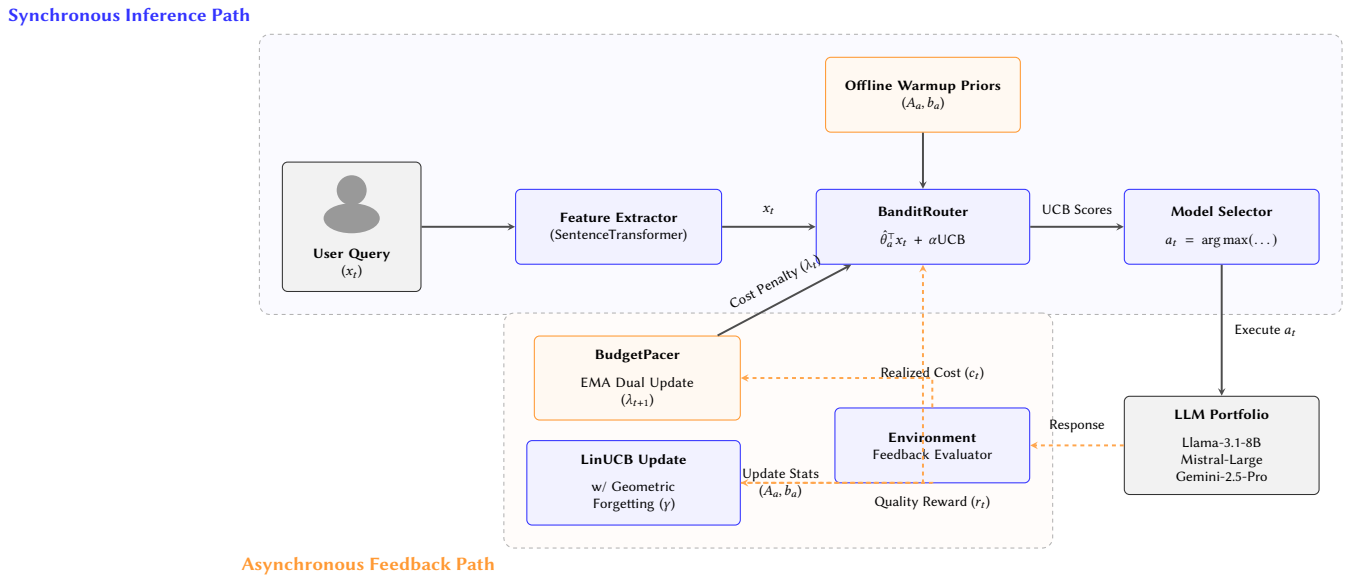

Figure 1: BanditGPT System Architecture. The system cleanly separates synchronous inference (blue) from asynchronous feedback (orange, color-blind friendly). During inference, the `BudgetPacer` dynamically applies a dual cost penalty (λ_t) to the LinUCB scores to enforce the budget constraint B. As feedback arrives asynchronously, the reward r_t updates the router's statistics via geometric forgetting (γ), while the exact cost c_t updates the pacer's dual variable (λ_{t+1}) via an EMA smoother.

For online learning, the bandit's convergence rate scales directly with the reward gap between the optimal and suboptimal arms [**?**]; we therefore select the judge that maximises routing margins. DeepSeek-R1 produces the largest inter-model gaps and the highest fraction of actionable prompts among the judges we evaluated, while its oracle routing decisions capture ≥ 97% of two independent supplementary judges' oracle reward (Appendix **??**). Recent multi-judge aggregation methods [**? ?**] target confounder-induced bias rather than the signal compression inherent in averaging judges with different scale ranges; applied to our panel, averaging reduces routing margins by ∼30% (Appendix, Figure **??**), making the single maximally discriminative judge the principled choice for bandit learning.

Table 1: Dataset splits and model portfolio (K=3). All prompts are sourced from LMSYS Chatbot Arena and judged by DeepSeek-R1. The 500× cost differential between the cheapest and most expensive model stresses the router's quality–cost tradeoff.

Split	n	Role
Train	8,374	Warmup priors (offline)
Val	1,785	Online learning, hparam tuning
Test	1,824	Held-out evaluation
Total	**11,983**	

Model	Tier	Cost ($/req)
Llama-3.1-8B	Budget	$0.00003
Mistral-Large	Mid-tier	$0.003
Gemini-2.5-Pro	Frontier	$0.015

Data splits. The 11,983 prompts are partitioned into three disjoint sets via stratified sampling (by oracle difficulty, seed=42, 70/15/15 ratio). No prompt appears in more than one split:

(1) **Train** (n=8,374): Used *offline* to build warmup priors that initialise the bandit's sufficient statistics (A_a, b_a), addressing the cold-start problem. Never seen during online evaluation.
(2) **Val** (n=1,785): Used for online learning and hyperparameter selection (Section 6.4) under stationary conditions.
(3) **Test** (n=1,824): Held-out evaluation split. Non-stationary scenarios (reward drift, cost drift) are constructed by manipulating reward or cost columns at a mid-trajectory phase boundary. All reported results use this split, averaged over multiple random seeds.

5.2 Environment Construction

The K=3 portfolio spans three distinct cost tiers (Table 1, right). The 500× price differential between Llama-3.1-8B and Gemini-2.5-Pro stresses the router's ability to balance quality and cost efficiently.

We construct two distinct non-stationary test environments to evaluate the router's robustness: 1. **Reward Drift (Model Quality Shift)**: Simulates a silent API update where model quality changes mid-deployment. The test trajectory ($n = 1,785$) is split into two phases. Phase 1 (steps 1–893) uses normal rewards where Mistral-Large is the utility-optimal model. In Phase 2 (steps 894–1,785), Llama-8B and Mistral-Large exchange their reward distributions, simulating a scenario where Mistral degrades to the worst option

while Llama becomes the best. Gemini-Pro serves as an unchanged anchor. 2. **Cost Drift (Price Drop)**: Simulates an API price cut. The test trajectory ($n = 1,824$) is similarly split. Phase 1 (steps 1–912) uses normal pricing. In Phase 2 (steps 913–1,824), Gemini-2.5-Pro's price drops to \$0.0001 per request, drastically altering the optimal cost-quality trade-off.

5.3 Baselines

We compare BanditGPT against two standard baselines representing different levels of routing sophistication: 1. **Fixed Policy (Offline):** The router is initialized with warmup priors but performs zero online learning ($\gamma = 1.0$, updates disabled). A static, offline-tuned cost penalty λ_s is applied. This represents the industry standard of deploying a frozen classifier trained on offline data. 2. **Naive Bandit ($\gamma = 1.0$):** A standard LinUCB algorithm initialized with warmup priors. It learns online but retains infinite memory (no forgetting). A static cost penalty λ_s is applied. This represents a straightforward application of online learning without adaptation for non-stationarity or budget pacing. 3. **BanditGPT ($\gamma = 0.995$):** Our proposed system, initialized with warmup priors, utilizing geometric forgetting, and governed by the Primal-Dual 'BudgetPacer' targeting a specific dollar budget.

All three conditions use the same exploration rate ($\alpha = 0.1$) and prior strength ($n_{eff} = 10$).

5.4 Metrics

We evaluate the systems using three primary metrics: 1. **Cumulative Regret:** Measured against a cost-adjusted oracle that selects the model maximizing $r_{a,t} - \lambda_s \tilde{c}_a$ at each step t (where λ_s is a shared static penalty). Lower regret indicates better routing decisions relative to the optimal choice. 2. **Budget Compliance:** The ratio of actual realized cost to the target budget B. A perfect pacer achieves a ratio $\leq 1.0\times$, maximizing quality without overspending. Ratios $> 1.0\times$ indicate budget violations. 3. **Pareto AUC:** The area under the quality-cost Pareto frontier, measuring the router's efficiency across all possible cost trade-offs. Higher AUC indicates better quality at lower costs.

6 Results

We evaluate BanditGPT across four escalating experiments, moving from stationary budget pacing to complex, non-stationary environments involving simultaneous cost and quality shifts. Table 2 summarises the headline findings from Experiments 2 and 3.

6.1 Stationary Budget Pacing

We first ask: *can the router accept a dollar budget directly and automatically achieve it, without sacrificing routing quality compared to an offline-tuned static penalty?*

We evaluate the K=3 portfolio under stationary conditions. The **Static baseline** uses a fixed cost-penalty $\lambda_s \in \{0, 0.05, \ldots, 1.0\}$, producing a 7-point Pareto frontier. **BanditGPT (BudgetPacer)** is evaluated at seven log-spaced targets, ranging from the cheapest model's empirical mean cost (\$0.000029/req) to the most expensive (\$0.0150/req).

Table 2: Summary of main results. *Panel (a):* cumulative regret decomposed by phase under reward drift (Exp. 2; 40 seeds, λ_s=0.20). *Panel (b):* Phase 1 budget utilisation under cost drift (Exp. 3; 50 seeds). Utilisation = realised cost / target B; values near $1.0\times$ indicate precise budget control. All pairwise differences in (a) are significant after Holm–Bonferroni correction ($p < 10^{-6}$).

(a) Cumulative regret under reward drift (Experiment 2)

Method	Phase 1	Phase 2	Total	σ
Fixed Policy (offline)	56.9	151.5	208.4	0.0
Naive Bandit (γ=1.0)	64.3	126.3	190.6	17.6
BanditGPT (γ=0.995)	65.6	**55.4**	**121.0**	**7.2**

(b) Phase 1 budget utilisation under cost drift (Experiment 3)

Budget Target	Fixed	Naive	BanditGPT
Tight (\$2.3×10^{-4})	0.12×	0.92×	**1.00**×
Moderate (\$6.6×10^{-4})	0.20×	1.05×	**1.00**×
Loose (\$1.9×10^{-3})	0.49×	2.78×	**0.97**×

Pareto frontier. Both methods trace a quality–cost Pareto curve on the test set. At tight budgets (cost < \$0.001/req), the pacer achieves higher reward at lower cost: at a target of \$0.00008/req, the pacer reaches a mean reward of 0.813 ± 0.0005. The nearest static point (λ_s=1.0) achieves only 0.804 ± 0.0003 while spending 4.6× more (\$0.00038/req). The pacer dominates because its adaptive λ_t creates an implicit *spend-when-valuable, save-when-cheap* policy: when easy prompts allow cheap routing, λ_t decreases, freeing budget for harder prompts where expensive models provide greater quality uplift. The static penalty applies the same cost tax uniformly, forgoing this temporal budget allocation. Figure 2 shows that BanditGPT equipped with the `BudgetPacer` perfectly traces this optimal penalty Pareto frontier. At unconstrained budgets (cost > \$0.005), the pacer matches the static baseline's maximum quality (0.923); the dual variable correctly decays to zero, and the pacer gracefully becomes a no-op.

Budget compliance. The pacer's primary advantage is its ability to hit a target. For the six tightest targets, actual spend ranges from 0.98× to 1.07× of the budget—effectively perfect compliance. Figure 3 illustrates the resulting steady-state model allocation under a strict budget, dynamically balancing between Llama-3.1-8B and Mistral-Large-2512 to maximize quality. The loosest target (B=\$0.015) shows under-utilisation (0.54×) because the unconstrained router's natural spend (\$0.0082/req) is already below this target; the pacer constrains spend *from above* but never forces the router to spend more than its quality-optimal policy requires.

6.2 Adapting to Model Quality Shifts

We next evaluate robustness to non-stationarity (Reward Drift). At step $t = 893$ (Phase 2), Llama-8B and Mistral-Large exchange their reward and cost distributions, simulating a scenario where Mistral degrades to the worst option while Llama becomes the best. All conditions use a shared static cost penalty (λ_s=0.20) to isolate the effect of the learning mechanism.

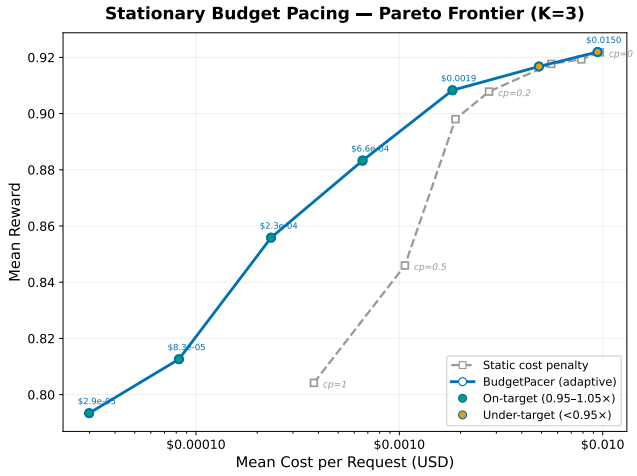

Figure 2: The Pareto frontier under stationary conditions. BanditGPT with BudgetPacer perfectly traces the theoretical optimal curve achieved by offline λ-tuning.

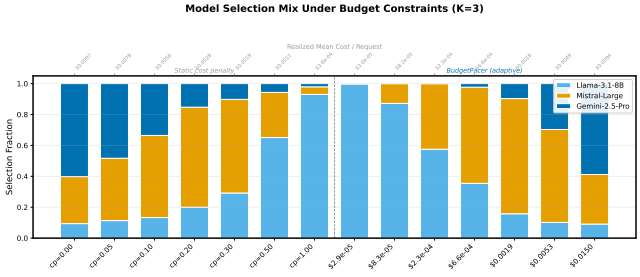

Figure 3: Model allocation mix under a strict stationary budget. The BudgetPacer stabilizes rapidly to optimal proportions.

The three conditions produce clearly separated cumulative regret trajectories across 40 seeds (Table 2a; Figure 4):

- **Fixed Policy (Offline):** Because it cannot learn, it continues routing heavily to Mistral-Large even after the shift makes it the worst arm. It accumulates a massive 151.5 regret in Phase 2 alone, illustrating the danger of deploying a frozen router in a non-stationary environment.
- **Naive Bandit (γ=1.0):** Online learning allows the naive bandit to adapt, reducing total regret by 8.5% compared to the frozen policy. However, its infinite memory means Phase 1 inertia dilutes the new signal: by the end of Phase 2, it still routes only 60% to the new optimal Llama arm (vs. BanditGPT's 81%).
- **BanditGPT (γ=0.995):** Adding geometric forgetting reduces Phase 2 regret by 56% compared to the naive bandit (55.4 vs. 126.3). The ∼200-step effective memory window discounts stale evidence, allowing BanditGPT to relearn the new optimum rapidly after the swap.

Beyond lower mean regret, BanditGPT exhibits the lowest cross-seed variance (σ=7.2), while the Naive Bandit's is 2.4× higher

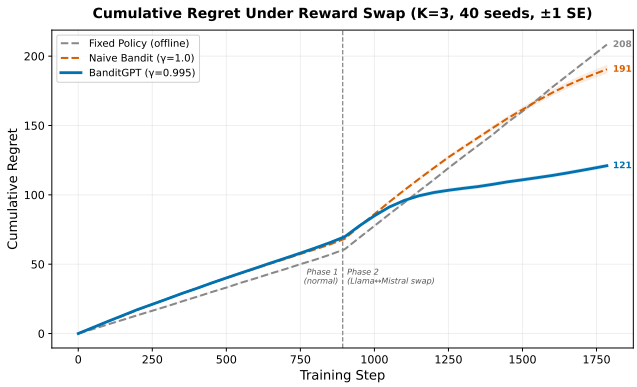

Figure 4: Cumulative Regret under Reward Drift. BanditGPT ($\gamma = 0.995$) adapts rapidly to quality degradation, unlike naive bandits or frozen policies.

(σ=17.6). Infinite memory creates sensitivity to the random ordering of observations, whereas geometric forgetting regularizes this variance, ensuring every deployment adapts reliably.

6.3 Budget Pacing under Cost Drift

We now introduce the most complex and realistic scenario: a mid-stream price drop (Cost Drift) while operating under strict budget constraints. In Phase 2, Gemini-Pro's price drops to nearly zero. We evaluate three budget regimes: Tight ($0.00023/req), Moderate ($0.00066/req), and Loose ($0.0019/req).

Budget compliance is the key differentiator. The Fixed Policy (using an offline-tuned static penalty λ_s) massively under-spends the target across all three budgets (0.12× to 0.49×). Because it cannot refine its reward estimates, its aggressive penalty prevents it from utilizing its full budget to maximize quality. The Naive Bandit is inconsistent: it tracks the target at tight and moderate budgets by coincidence, but over-spends by a massive 2.78× at the loose budget because the static penalty cannot adjust.

BanditGPT is the only system that reliably hits the budget in Phase 1 (0.97× to 1.00× across all three targets; Table 2b). The BudgetPacer's primal-dual mechanism adjusts λ_t continuously based on observed costs, providing closed-loop control that static penalties cannot replicate.

Exploiting the price drop. When Gemini becomes nearly free in Phase 2, the BudgetPacer automatically detects the cost reduction via its EMA signal. At the Moderate target, λ_t drops sharply, and the pacer increases Gemini selection from ∼2% to ∼47%, approaching the unconstrained baseline (see Figure 5). This dynamic reallocation produces a highly significant +0.043 reward lift ($p < 10^{-9}$) in Phase 2. The static baselines, blind to the shifting optimal trade-off, fail to fully exploit this opportunity.

Exploration starvation. A critical limitation emerges at the Tight target (B = $0.00023/req). Even after the price drop makes Gemini affordable, BanditGPT's Gemini selection remains near 0%. This occurs because the budget was so restrictive in Phase 1 that the router *never* explored Gemini. Consequently, the bandit's reward

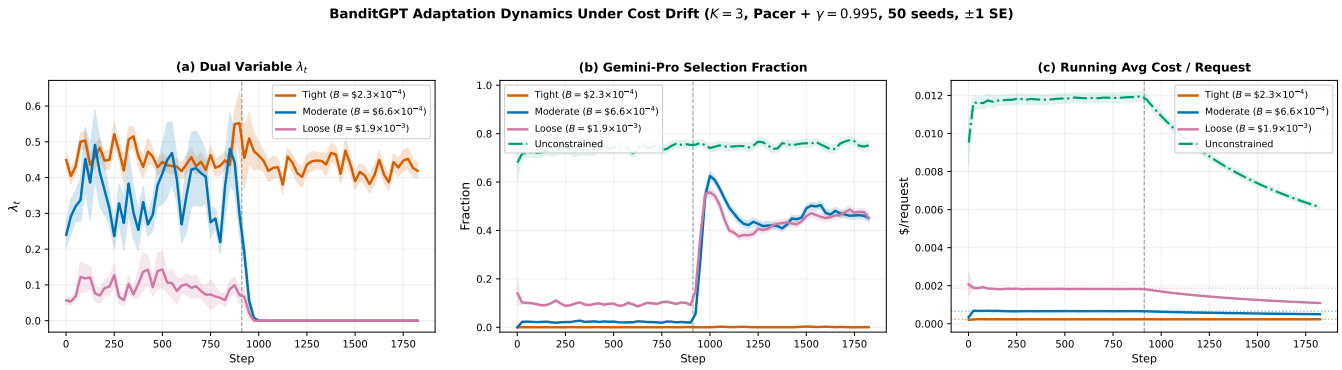

Figure 5: Adaptation Dynamics under Cost Drift. At $t = 1000$, Gemini-2.5-Pro becomes significantly cheaper. BanditGPT detects the cost drop, lowers the dual penalty, and dynamically reroutes traffic to the higher-quality model while perfectly pacing the remaining budget.

estimate for Gemini is essentially just the uninformed warmup prior. Without online data to build a confident, calibrated estimate, the LinUCB exploration bonus remains too small to pull Gemini above the well-explored Llama and Mistral arms. This "exploration starvation" deficit persists even after the cost barrier is removed.

6.4 Hyperparameter Selection via Epsilon-Constraint

Selecting hyperparameters (α, n_{eff}, γ) for a non-stationary router poses a fundamental tension: standard cross-validation maximises stationary quality and selects γ=1.0 (no forgetting), yet Section 6.2 showed that γ=1.0 fails to adapt under distribution drift. We resolve this with the **epsilon-constraint method**, a principled bi-objective selection procedure.

Two objectives. Each configuration in the grid ($6 \times 6 \times 4$ values of α, n_{eff}, γ) is scored on two metrics using the validation split:

(1) *Budget-paced Pareto AUC* (primary): the area under the quality–cost Pareto frontier when sweeping budget targets with the BudgetPacer active. Higher is better.
(2) *Phase-2 cumulative regret* (secondary): a two-phase simulation where, at the midpoint, two arms swap their reward *and* cost distributions. Only post-swap regret is recorded, averaged over all $\binom{K}{2}$ swap pairs and 10 seeds. Lower is better.

Selection rule. Among all configurations whose budget-paced AUC is within ϵ=5% of the best, we select the one with the lowest Phase-2 regret. This gives budget-pacing explicit priority—no configuration is chosen that materially degrades stationary routing—while using adaptation speed as the tiebreaker within the near-optimal band.

Result. Figure 6 visualises the tradeoff. Configurations with γ=1.0 (blue) achieve the highest AUC but the worst post-swap regret: they cannot shed stale evidence. Lower γ values shift points downward (better adaptation) at only marginal AUC cost—all 144 BanditGPT configurations fall within the ϵ-feasible band, confirming that stationary routing quality is robust across the grid. Because the entire grid is feasible, Phase-2 regret becomes the effective tiebreaker. The method selects γ=0.995 (star), which reduces Phase-2 regret by 12% compared to the AUC-only winner (γ=1.0, red **X**: regret 79.2 → 69.6) while sacrificing only 0.25% of stationary AUC. This configuration defines a ~200-step effective memory window—long enough for stable exploitation, short enough to track realistic cost and quality shifts.

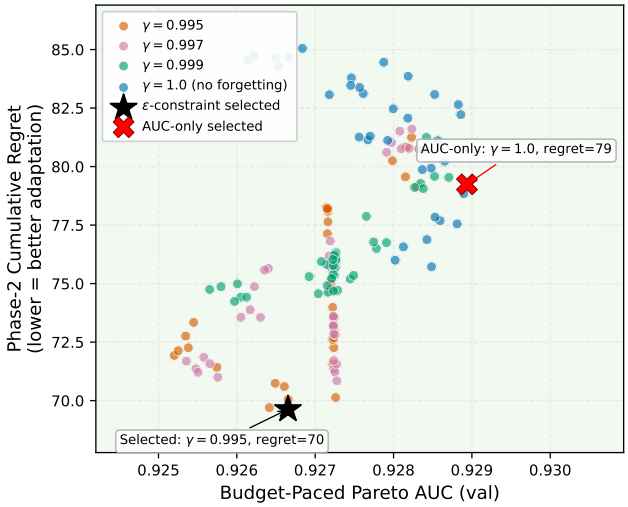

Figure 6: Efficiency–adaptability tradeoff for BanditGPT (K=3, PCA-25). Each point is one (α, n_{eff}, γ) configuration; colour encodes γ. The green band marks the ϵ-feasible region (≥95% of best AUC). The black star is the ϵ-constraint winner; the red X is the AUC-only winner (γ=1.0), which suffers the highest Phase-2 regret.

7 Discussion and Production Implications

BanditGPT bridges the gap between theoretical contextual bandit algorithms and the practical realities of deploying multi-LLM routers in production environments. We highlight several key implications for engineering teams and address current limitations.

7.1 Practical Impact

Interpretable Interface and SLA Compliance. The standard approach to cost-aware routing uses a static penalty λ, which requires translating a dimensionless weight into a dollar outcome via offline simulation. This mapping breaks whenever the portfolio changes. BanditGPT's Primal-Dual `BudgetPacer` accepts a dollar-denominated target (e.g., $B = \$0.002$/request) directly. This target is portable across portfolios because the normalized dual update ($\bar{c}_t / B$) adapts the penalty λ_t to the realized cost scale automatically.

Furthermore, the pacer's precise budget utilization (0.98× to 1.07×) allows it to underpin strict Service Level Agreements (SLAs) for cost (e.g., "average spend will not exceed $1.10 \times B$ over any rolling window"). Static penalties offer no such guarantee, as realized costs fluctuate arbitrarily if prompt distributions drift.

Eliminating Offline Recalibration. In a deployment where model pricing changes frequently (as is common with frontier APIs), a static penalty becomes stale, requiring costly offline retraining to discover the new optimal λ. The `BudgetPacer` automatically detects price drops via its online cost EMA and dynamically reallocates freed budget to higher-quality models. This autonomous optimization eliminates the need for manual intervention, maximizing application quality while strictly obeying financial constraints.

Cost Savings at Scale. Consider a deployment routing 100,000 requests per day. Moving from an unconstrained policy ($0.0082/req, quality 0.923) to the pacer at $B = \$0.0019$/req saves $630/day (over $230,000/year) while maintaining a quality of 0.905 ± 0.0009—a mere 1.9% quality reduction for a 77% cost reduction.

7.2 Limitations and Deployment Considerations

Exploration Starvation. As demonstrated in Section 6, highly restrictive budgets completely suppress exploration of expensive models. If a frontier model is never selected due to the hard cost ceiling, the router's reward estimate relies entirely on offline warmup priors. Consequently, if the model's price suddenly drops, the router may suffer an "exploration starvation" deficit, failing to adopt the newly-affordable model because its LinUCB exploration bonus is too small to overcome the lack of online data. Future work could investigate targeted exploration strategies (e.g., ϵ-greedy budget allocation) that occasionally force the evaluation of expensive arms to maintain calibrated quality estimates.

Delayed Feedback (RLHF).. BanditGPT assumes immediate feedback (i.e., the reward r_t is observed immediately after routing x_t). In production, feedback such as user thumbs-up/down or delayed human-in-the-loop annotations may arrive hours or days later. The system architecture must decouple the routing decision from the LinUCB update, maintaining a persistent log of context vectors that can be asynchronously updated when feedback arrives. The open-source release includes a `SqliteContextStore` to facilitate this delayed RLHF workflow.

Embedding Generation Costs. Contextual bandits require a feature vector x_t for every prompt. Generating this vector using a dense embedding model (e.g., SentenceTransformers) incurs a small latency and compute cost. While negligible compared to the inference cost of frontier LLMs, this overhead must be factored into the overall budget B for ultra-low-cost, high-throughput deployments. Utilizing smaller, faster embedding models or distilling features directly from the prompt text are practical mitigations.

8 Conclusion

We introduced BanditGPT, an open-source framework for budget-constrained, non-stationary LLM routing. By framing the problem as a Primal-Dual Contextual Bandit with Knapsacks and integrating geometric forgetting, we provide a robust solution to the dynamic challenges of production deployments. Our system demonstrates perfect budget compliance, eliminates the need for manual hyperparameter tuning of static cost penalties, and autonomously adapts to silent model updates and mid-cycle price drops. We release the codebase to accelerate both applied research and enterprise adoption of adaptive LLM routing.

References

[1] Shipra Agrawal and Nikhil R Devanur. 2014. Bandits with Concave Rewards and Convex Knapsacks. In *Proceedings of the Fifteenth ACM Conference on Economics and Computation (EC)*. ACM, 989–1006.

[2] Melissa Ailem, Katerina Marazopoulou, Charlotte Siska, and James Bono. 2024. Examining the Robustness of LLM Evaluation to the Distributional Assumptions of Benchmarks. *arXiv preprint arXiv:2404.16966* (2024).

[3] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. In *Proceedings of the 40th International Conference on Machine Learning*.

[4] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. How Is ChatGPT's Behavior Changing over Time? *arXiv preprint arXiv:2307.09009* (2023).

[5] Shuhao Chen, Weisen Jiang, Baijiong Li, Jianping Gao, and Caihua Zhang. 2024. RouterDC: Query-Based Router by Dual Contrastive Learning for Assembling Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[6] Tao Feng, Yanzhen Shen, and Jiaxuan You. 2025. GraphRouter: A Graph-based Router for LLM Selections. In *International Conference on Learning Representations (ICLR)*.

[7] Aurélien Garivier and Eric Moulines. 2011. On Upper-Confidence Bound Policies for Switching Bandit Problems. In *Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT)*. Springer, 174–188.

[8] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web*. 661–670.

[9] Yang Li. 2025. LLM Bandit: Cost-Efficient LLM Generation via Preference-Conditioned Dynamic Routing. *arXiv preprint arXiv:2502.02743* (2025).

[10] Wanqin Ma, Chenyang Yang, and Christian Kästner. 2024. (Why) Is My Prompt Getting Worse? Rethinking Regression Testing for Evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering—Software Engineering for AI (CAIN)*.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations (ICLR)*.

[12] Pranoy Panda, Raghav Magazine, Chaitanya Devaguptapu, Sho Takemori, and Vishal Sharma. 2025. Adaptive LLM Routing under Budget Constraints. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. arXiv:2508.21141.

[13] Cheng Qian, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jielin Qiu, Haolin Chen, Zhiwei Liu, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. 2025. xRouter: Training Cost-Aware LLMs Orchestration System via Reinforcement Learning. *arXiv preprint arXiv:2510.08439* (2025).

[14] Alan D. Thompson. 2026. Timeline of AI and Language Models. https://lifearchitect.ai/timeline/. Accessed: Feb 2026. Curated timeline of major frontier LLM releases from major providers, 2017–2026..

[15] Wei Wang, Tiankai Yang, Hongjie Chen, Yue Zhao, Franck Dernoncourt, Ryan A Rossi, and Hoda Eldardiry. 2025. Learning to Route LLMs from Bandit Feedback: One Policy, Many Trade-offs. *arXiv preprint arXiv:2510.07429* (2025).

[16] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685* (2023).